

Car Collision Detection System

**A PROTOTYPE FOR AUTOMOTIVE SAFETY USING
ARDUINO**

Nitish Bhosle (D15A/04)

Aryan Dangat (D15A/12)

Date: 17/04/25

Subject: Wireless Technology (WT)

Introduction

- **Objective: Enhance vehicle safety by detecting collisions and displaying location data.**
- **Key Components:**
 - **Arduino Uno: Microcontroller**
 - **MPU6050: Motion sensor**
 - **NEO-6M GPS: Location tracking**
 - **I2C LCD: Status display**
 - **Breadboard: Prototyping platform**

COMPONENTS OVERVIEW

- Arduino Uno: Processes sensor data and coordinates the system.
- Breadboard: Connects components without soldering.
- MPU6050: Detects sudden motion changes (accelerometer & gyroscope).
- NEO-6M GPS Module: Provides latitude and longitude.
- I2C LCD: Shows "System Ready" or collision alerts.

MPU6050 ACCELEROMETER & GYROSCOPE

```
Adafruit_MPU6050 mpu;  
// In setup()  
:if (!mpu.begin()) {  
    Serial.println("Failed to find MPU6050");  
    lcd.setCursor(0, 1);  
    lcd.print("MPU6050  Error");  
    while(1);  
}  
mpu.setAccelerometerRange(MPU6050_RANGE_8_G);  
mpu.setGyroRange(MPU6050_RANGE_500_DEG)  
;mpu.setFilterBandwidth(MPU6050_BAND_21_HZ);
```



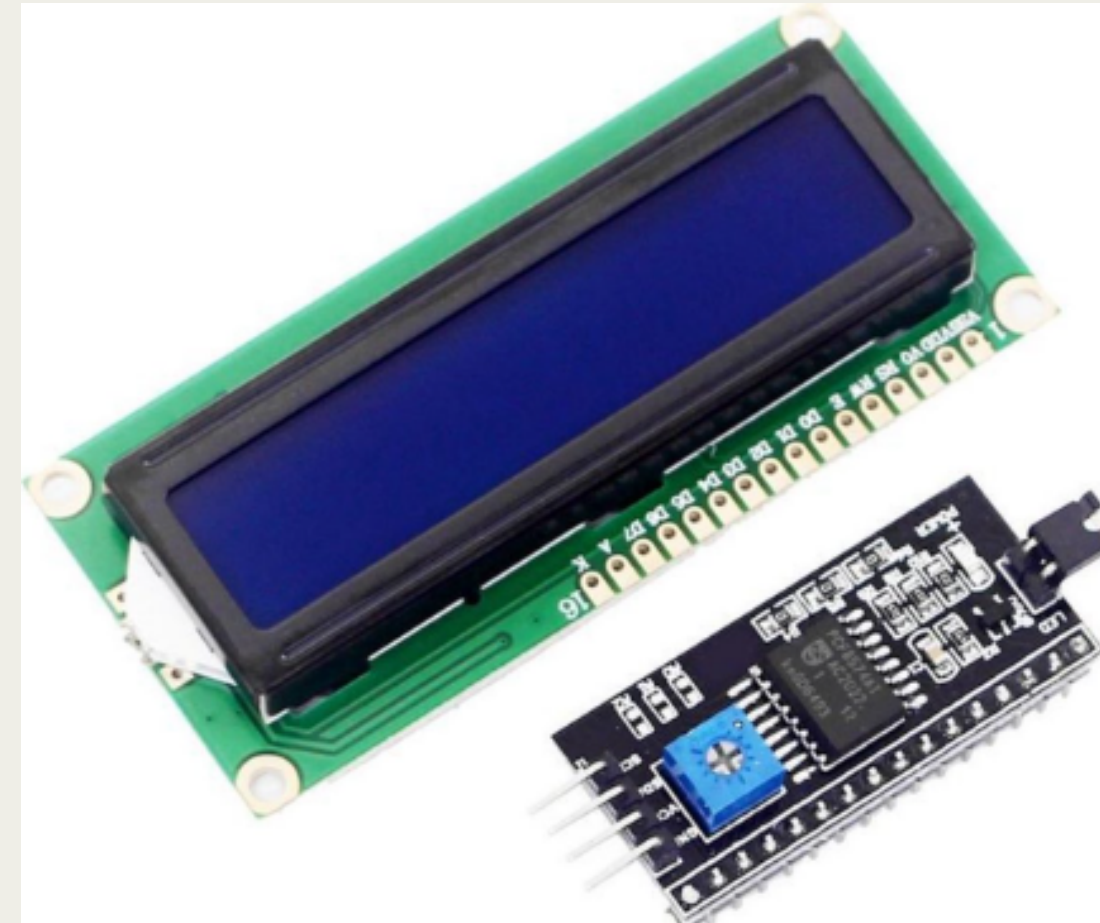
The MPU6050 is initialized using the Adafruit library. The initialization process:

1. Creates an instance of the Adafruit_MPU6050 class
2. Calls mpu.begin() to establish I2C communication with the sensor
3. Configures the accelerometer range to $\pm 8g$, providing sufficient range to detect collision forces
4. Sets the gyroscope range to ± 500 degrees per second
5. Configures the digital low-pass filter bandwidth to 21 Hz, which helps reduce noise while maintaining responsiveness
6. If initialization fails, displays an error message on the LCD and enters an infinite loop to halt further execution

I2C LCD DISPLAY

```
// Initialize LCD (address 0x27, 16x2 display)
LiquidCrystal_I2C lcd(0x27, 16, 2);

// In setup():
Wire.begin();
lcd.begin();
lcd.backlight();
lcd.setCursor(0, 0);
lcd.print("System Ready");
```



The LCD display initialization involves:

1. Creating an instance of `LiquidCrystal_I2C` with the I2C address (0x27) and display dimensions (16 columns × 2 rows)
2. Initializing the I2C bus with `Wire.begin()`
3. Initializing the LCD with `lcd.begin()`
4. Turning on the backlight for visibility
5. Setting the cursor to the first position (0,0) and displaying the initial "System Ready" message

NEO-6M GPS MODULE

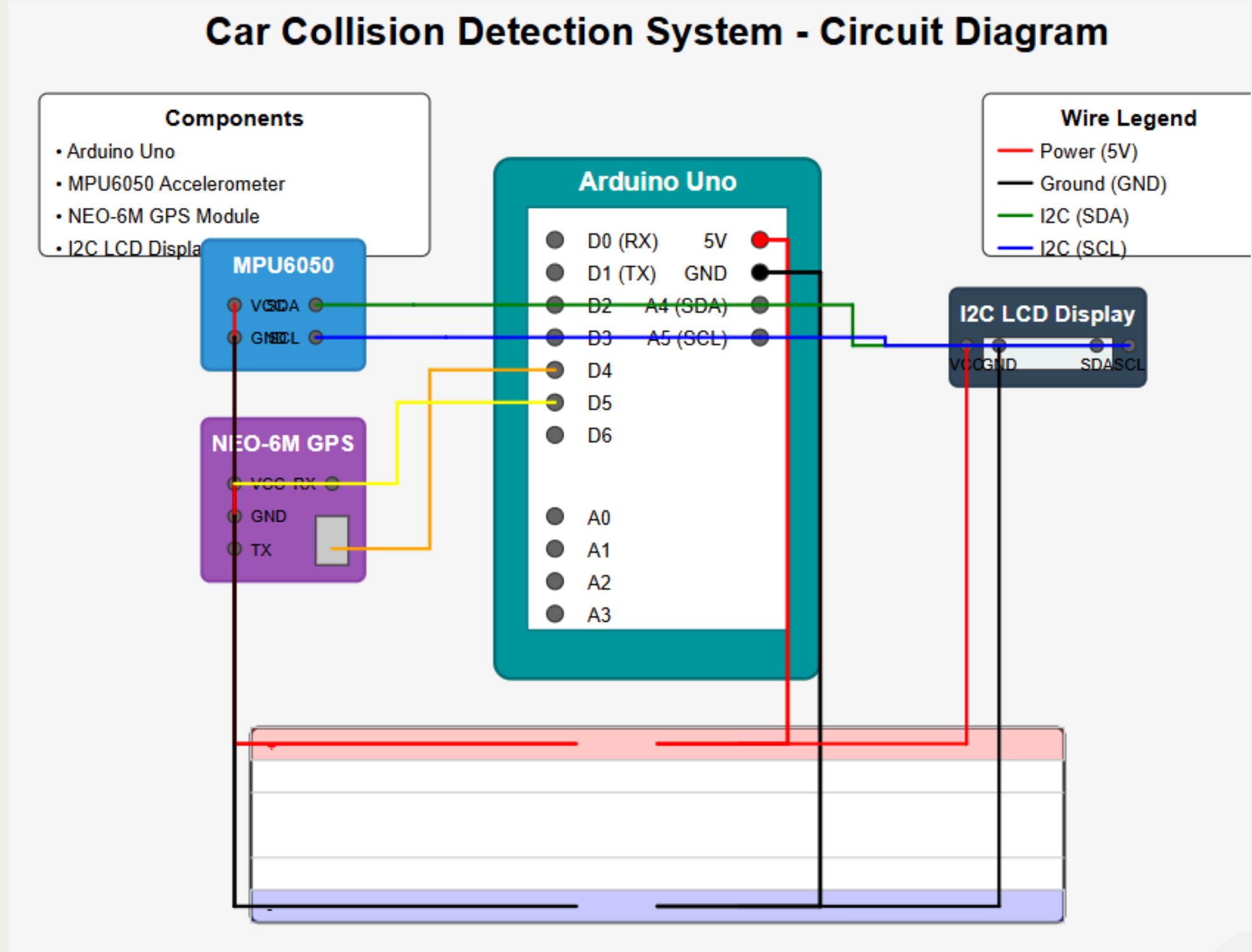
```
// Initialize GPS (RX, TX pins)
SoftwareSerial gpsSerial(4, 5);
TinyGPSPlus gps;
// In setup():
gpsSerial.begin(9600);
```



The GPS module is initialized through:

- Creating a software serial instance using pins 4 (RX) and 5 (TX) for communication
- Creating an instance of the TinyGPSPlus class to parse the NMEA data
- Setting the baud rate to 9600 bps, which is the standard rate for most NEO-6M GPS modules
- The TinyGPS++ library handles the parsing of GPS data in the main loop when data is available

CONNECTION DIAGRAM

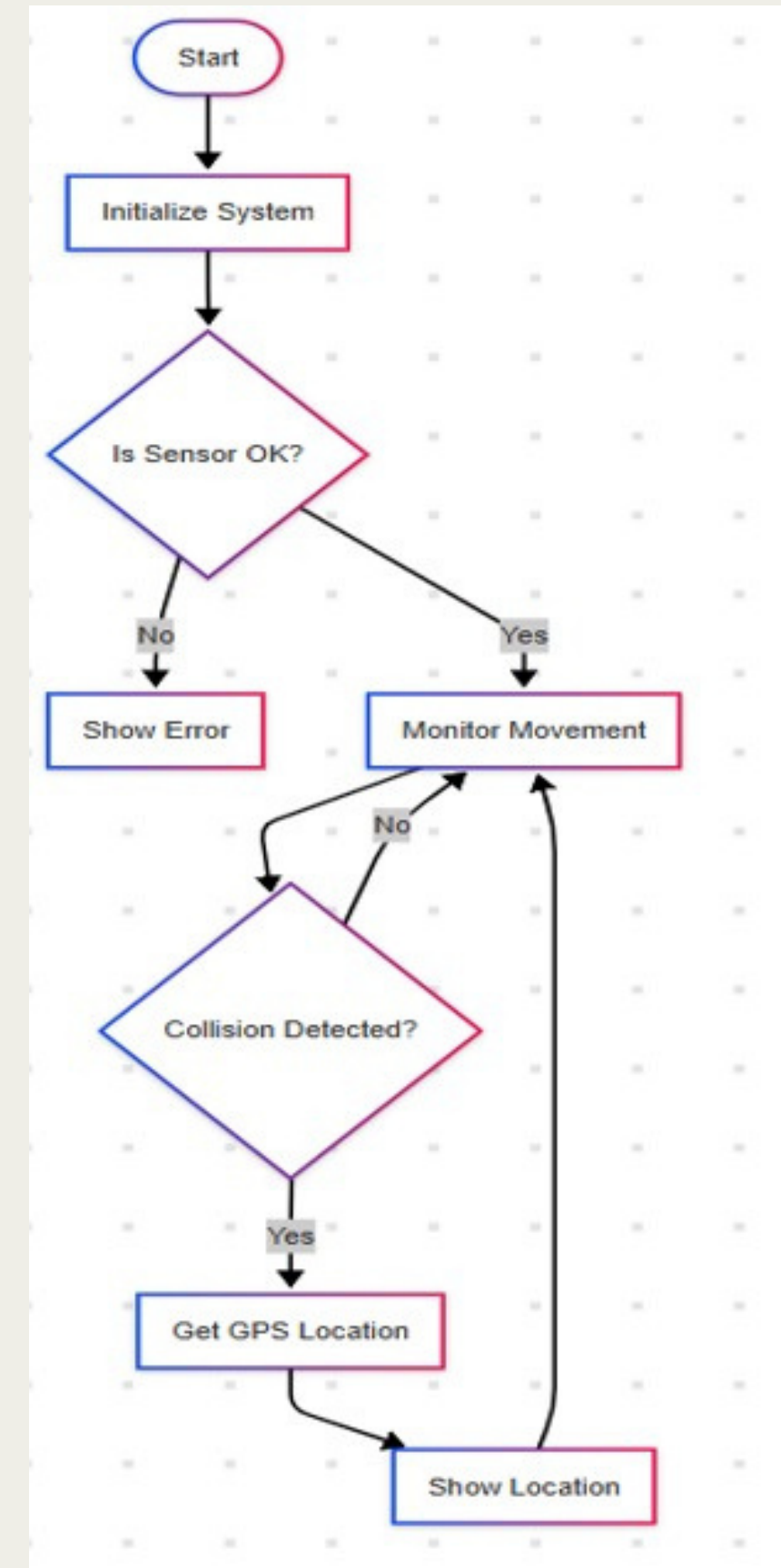


BLOCK DIAGRAM

This flowchart represents a car accident detection system using sensors and GPS. Here's a brief description:

1. System Initialization occurs first.
2. It checks if the sensor is functioning properly.
 - If not, it shows an error.
3. If the sensor is OK, it monitors movement.
4. If a collision is detected, it:
 - Retrieves the GPS location.
 - Then displays the location.
5. If no collision is detected, it continues monitoring.

This logic ensures that accidents are detected and reported



SYSTEM FUNCTIONALITY

- Process:
 - a. MPU6050 monitors acceleration.
 - b. Arduino calculates magnitude.
 - c. If $> 20 \text{ m/s}^2$, collision detected.
 - d. Retrieves GPS coordinates.
 - e. LCD shows "Collision Detected" + location.
 - f. Else, "System Ready."
- Real-Time: Immediate collision response.

CONCLUSION

- **Success:** Integrated sensors and GPS for collision detection.
- **Outcome:** Real-time alerts with location data.
- **Future Enhancements:** Audible alerts, data logging.